

Arrays, Listas e Tuplas



Variáveis



Diferença entre Variáveis Simples e Compostas

As **variáveis simples** só armazenam um elemento por vez. Caso precise colocar outro valor deverá remover o item anterior.



Variável simples

As **variáveis compostas**, armazenam vários elementos de uma vez, através de espaços que ela criará dentro de uma única variável.



Variável composta

Diferença entre Variáveis Simples e Compostas

Simples

variável = 1

```
# Variável simples  
numero = 10  
print(numero) # Saída: 10
```

Composta

variável = [1, 3, 8, 9]

```
# Variável composta (lista)  
numeros = [10, 20, 30]  
print(numeros) # Saída: [10, 20, 30]
```

Arrays (vetores)



Array (vetores)

- Um **vetor** (também conhecido como **array**) é uma **estrutura de dados** que armazena **múltiplos valores** de um mesmo tipo em uma única variável.
- Vetores permitem **acessar** e **manipular** elementos de maneira rápida usando seus índices.
- Vetores são úteis quando precisamos trabalhar com uma **grande quantidade de dados**, especialmente quando esses dados têm o mesmo tipo.
- Ao invés de criar várias variáveis independentes, podemos **agrupar os dados em uma única variável** usando um vetor. Isso torna o código mais eficiente e organizado.

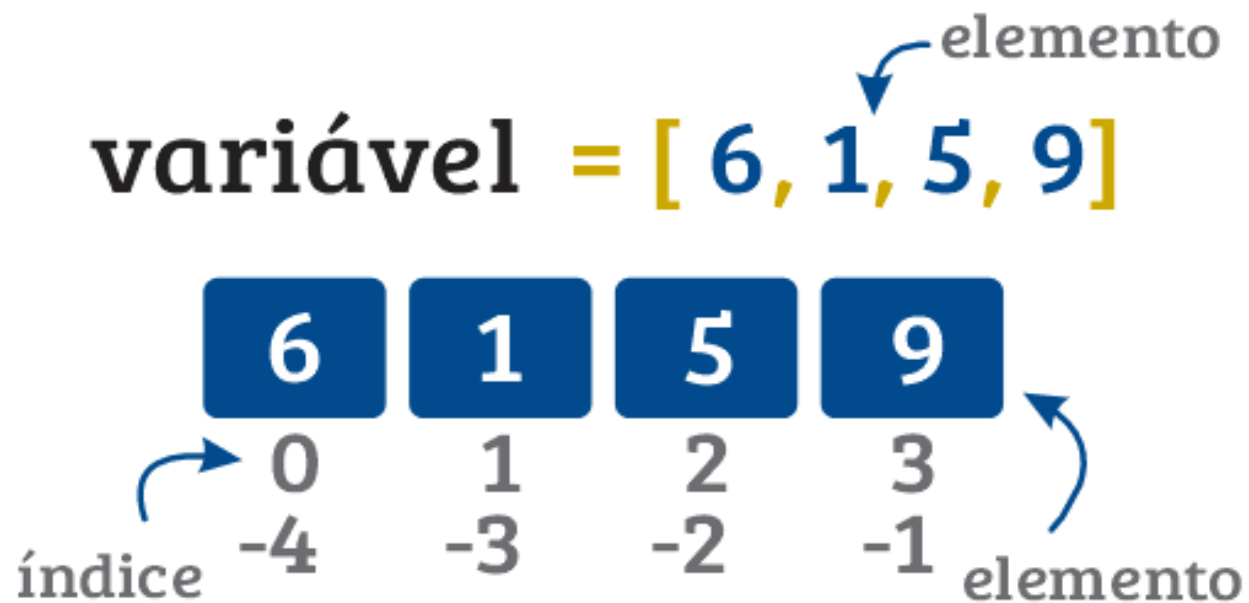
variável = **[]** (vazia)

variável = **[6, bola, 5.3, True]**

variável = **6** **'Bola'** **5.3** **True**

Array (vetores)

- Cada valor em um vetor é chamado de **elemento** e cada elemento é identificado por uma **posição específica** no vetor, chamada de **índice**.
- Os **índices** em um vetor começam em 0. O primeiro elemento está na posição 0, o segundo na posição 1, e assim por diante.
- **Índices** também podem ser **negativos**, o que permite acessar os elementos de trás para frente.



Por que Usar Vetores?

- Aqui está um exemplo simples de vetor em Python usando uma lista (já que Python não possui arrays como em outras linguagens, o conceito de vetor é implementado através de listas):

```
# Criando um vetor de números
vetor_numeros = [10, 20, 30, 40, 50]

# Acessando o primeiro elemento (índice 0)
print(vetor_numeros[0]) # Saída: 10

# Acessando o último elemento (índice -1)
print(vetor_numeros[-1]) # Saída: 50
```

- Organização:** Em vez de criar várias variáveis, um vetor permite armazenar tudo em um só lugar.
- Eficiência:** Usar vetores reduz a necessidade de escrever código repetitivo.
- Facilidade de Acesso:** Podemos acessar qualquer elemento do vetor de forma direta usando o índice.

```
print(vetor_numeros[-1]) # Saída: 50
```


Exemplo de Vetores

- Imagine que você deseja armazenar as notas de 5 alunos. Em vez de criar 5 variáveis separadas para cada nota, você pode criar um vetor para armazená-las:

```
# Vetor para armazenar as notas de 5 alunos
notas = [8.5, 7.2, 9.0, 6.4, 7.8]
# Acessando a nota do terceiro aluno
print(f"A nota do terceiro aluno é: {notas[2]}")
```

Índice

0	1	2	3	4
---	---	---	---	---

```
print(f"A nota do terceiro aluno é: {notas[2]}")
```

Console

A nota do terceiro aluno é: 9.0

Permite
adicionar
e remover
elementos

Listas

☒	Ordenada
☐	Mutável
☐	Aceita duplicados

MUTÁVEL

Tuplas

☒	Ordenada
☐	Imutável
☐	Aceita duplicados

IMUTÁVEL

Não permite
adicionar e
remover
elementos

Listas



O que é uma Lista?

- Uma **lista** em Python é uma **estrutura de dados composta** que pode armazenar **múltiplos valores** em uma única variável.
- Diferente de arrays (em outras linguagens), uma lista pode armazenar **elementos de diferentes tipos** (números, strings, booleanos, etc.) e é **mutável**, o que significa que podemos alterar seus elementos depois de criá-la (adicionar, remover ou modificar elementos).

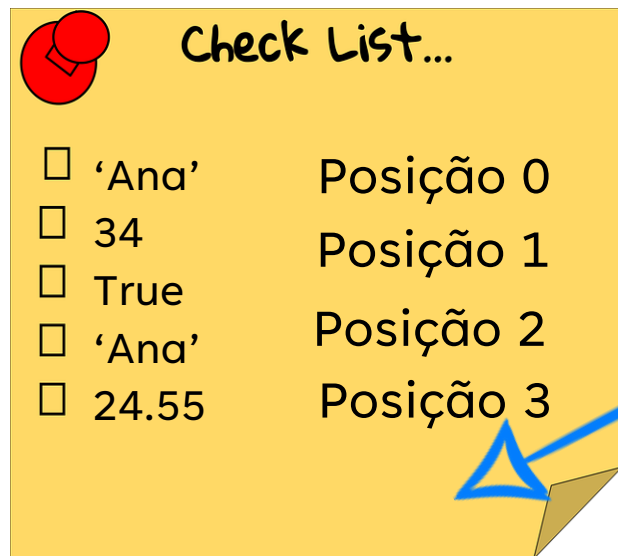
variável = **[]** (vazia)

variável = **[6, bola, 5.3, True]**

variável = **6** **'Bola'** **5.3** **True**

Características das Listas:

- **Mutabilidade:** Você pode modificar uma lista depois de criada.
- **Ordenação:** As listas mantêm os elementos na ordem em que foram inseridos.
- **Indexação:** Os elementos da lista podem ser acessados por seu índice (começando de 0).
- **Elementos Diversos:** Uma lista pode conter elementos de tipos diferentes (inteiros, strings, outras listas, etc.).



Se você adicionar **novos itens a uma lista**, os novos itens serão colocados no **final da lista**.

Como criar uma Lista?

Para criar uma lista é necessário você identificar com colchetes que está criando uma lista.

Exemplo

`lista = [ ]` → Criando uma lista vazia

`lista = ['Maria', 12, 1.56, True]` → Criando uma lista inicializada.
Lista pode receber vários tipos de dados.

`print(type(lista))`

Descobrimo o
tipo de dado
da variável

→ `<class 'list'>`

Acessando e Modificando Itens

```
# Criando uma lista de frutas
frutas = ['maçã', 'banana', 'laranja']
```

```
# Acessando o primeiro e o terceiro elemento
print(frutas[0]) # Saída: maçã
print(frutas[2]) # Saída: laranja

# Acessando o último elemento com índice negativo
print(frutas[-1]) # Saída: laranja
```

- Como as listas são **mutáveis**, podemos alterar seus valores diretamente atribuindo novos valores a determinados índices.

```
# Modificando o segundo item da lista
frutas[1] = 'uva'

print(frutas) # Saída: ['maçã', 'uva', 'laranja']
```

Adicionando Elementos: `append()`

- Podemos adicionar elementos em uma lista utilizando o método `append()`, que adiciona um novo elemento ao final da lista.

```
frutas = ['maçã', 'uva', 'laranja']  
  
# Adicionando um elemento ao final da lista  
frutas.append('morango')  
  
print(frutas) # Saída: ['maçã', 'uva', 'laranja', 'morango']
```

Inserir Elementos em uma Posição Específica: `insert()`

- O método `insert()` permite inserir um elemento em qualquer posição da lista.

```
# Inserindo um elemento na posição 1  
frutas.insert(1, 'pêra')  
  
print(frutas) # Saída: ['maçã', 'pêra', 'uva', 'laranja', 'morango']
```


Removendo Elementos: `remove()` ou `del`

- Para remover um elemento, podemos usar o método `remove()` ou a função `del`.

```
frutas = ['maçã', 'pêra', 'uva', 'laranja', 'morango']

# Removendo um elemento pelo valor
frutas.remove('laranja') # Remove a primeira ocorrência de 'laranja'

print(frutas) # Saída: ['maçã', 'pêra', 'uva', 'morango']

# Usando del para remover pelo índice
del frutas[0] # Remove o primeiro elemento

print(frutas) # Saída: ['pêra', 'uva', 'morango']
```

```
print(frutas) # Saída: ['pêra', 'uva', 'morango']
```

Tamanho da Lista: `len()`

- O número de elementos de uma lista pode ser obtido com a função `len()`.

```
print(len(frutas)) # Saída: 3
```

Estender lista: extend()

- Para anexar elementos de outra lista à lista atual, use o método `extend()`.

```
lista = ['Maria', 12]
lista2 = ['Paulo', 25]

lista.extend(lista2)

print(lista)

# Saída: ['Maria', 12, 'Paulo', 25]
```

```
# Saída: ['Maria', 12, 'Paulo', 25]
```

Remover da lista

- Existe três métodos de **exclusão** de elementos de uma lista:

1. `pop(posição): pop()`

- O método `pop()` é indicado quando você sabe o índice do elemento que procura. E ele retorna o índice removido, caso precise.

```
lista = ['Maria', 12]
```

```
print(lista.pop(1))
```

```
print(lista)
```

```
# Saída: ['Maria']
```

```
# Saída: ['Maria']
```

Item excluído

Se não especificar a posição,
ele remove o último da lista

Remover da lista

2. `del(posição): del()`

- O método `del()` é indicado quando não precisará do valor removido.

```
lista = ['Maria', 12]
```

```
del lista[1]
```

```
print(lista)
```

```
# Saída: ['Maria']
```

```
# Saída: ['Maria']
```

Item excluído

Remover da lista

2. `remove(item): remove()`

- O método `remove()` é indicado quando você sabe o `item` mas não o índice.

Item excluído

```
lista = [1, 2, 3, 4, 5]
lista.remove(3)
print(lista)

# Saída: [1, 2, 4, 5]
```



Recebendo dados do usuário e Percorrendo uma lista

Recebendo dados do usuário e adicionando dentro da lista através de um laço de repetição:

Verificando se está na lista

```
# Criando uma lista
lista = [1, 2, 3, 4, 5]

# Percorrendo todos os elementos da lista
for numero in lista:
    print(numero)
```

print(numero)

Adicionando em uma lista

```
# Criando uma lista vazia
lista = []

for indice in range(2):
    nome = input('Informe seu nome: ')
    lista.append(nome)

print(lista)
```

print(lista)

Exercícios



Exercício

Faça um programa que receba o nome, idade e a profissão de duas pessoas (utilize a estrutura de repetição for). Após isso, deve armazenar essas informações dentro de uma lista. E mostrar na tela o resultado.

Console

```
lista = []

for recebe in range(2):
    nome = input('Nome: ')
    idade = int(input('Idade: '))
    profissao = input('Profissão: ')

    lista.append(nome)
    lista.append(idade)
    lista.append(profissao)

print(lista)
```

```
Nome: Renata
idade: 38
prof: Programadora
Nome: Eduardo
idade: 37
prof: Programador
['Renata', 38, 'Programadora', 'Eduardo', 37, 'Programador']
```

`lista.append([nome, idade, profissao])`

`print(lista)`

Exercício

Faça um programa que verifique se certa palavra está dentro da lista. Peça pro usuário adicionar e verifique através do comando de verificação **in**.

```
lista = []

for recebe in range(2):
    nome = input('Nome: ')

    lista.append(nome)

print(lista)

print('ana' in lista)
```

```
nome = input('Qual o seu nome? ')
lista.append(nome)
```

```
Nome: Fabiana
Nome: Luana
['Fabiana', 'Luana']
False
```

```
lista = []

for indice in range(2):
    nome = input('Qual o seu nome? ')

    lista.append(nome)

nome2 = input('Qual o nome que deseja consultar: ')
print(f'O nome {nome2} faz parte da lista? {nome2 in lista}')
```

```
nome = input('Qual o seu nome? ')
lista.append(nome)
```

```
Qual o seu nome? Renata
Qual o seu nome? Luiza
Qual o nome que deseja consultar: Renata
O nome Renata faz parte da lista? True
```

Tuplas



- Tuplas são semelhantes às listas, mas são **imutáveis**. Isso significa que, uma vez criada, uma tupla não pode ser modificada (ou seja, você não pode alterar, adicionar ou remover elementos de uma tupla).
- Tuplas são úteis quando você deseja garantir que os dados não serão alterados.
- Tentar alterar uma tupla gerará um erro.
- Para criar um tupla usa-se parênteses. Ex.: `tupla = ()`

```
# Criando uma tupla
minha_tupla = (1, 2, 3, "Python", False)

# Acessando elementos da tupla
print(minha_tupla[0])  # Saída: 1
print(minha_tupla[3])  # Saída: Python
```

Adicionando elemento

Teríamos que **transformar a tupla em uma lista**, fazer a adição dos elementos e depois transformar em tupla novamente.

Tupla inicializada

Quero adicionar
um elemento

```
tupla = ('Maria', 12, 1.56, True)
```

```
lista = list(tupla)  
lista.append('novo valor')  
tupla = tuple(lista)
```

```
print(tupla)
```

```
# Saída: ('Maria', 12, 1.56, True, 'novo valor')
```

```
Tupla  
| ↓ list()  
Lista  
| ↓ append()  
Lista alterada  
| ↓ tuple()  
Nova Tupla
```

```
# Saída: ('Maria', 12, 1.56, True, 'novo valor')
```

Fatiamento de Tupla:

Tuplas suportam as mesmas operações de sequência que as listas, como **fatiamento**, **concatenação**, e **verificação de existência de elementos**, mas não suportam modificações diretas.

```
minha_tupla = (10, 20, 30, 40, 50)

print(minha_tupla[1:4]) # Acessa os elementos do índice 1 ao 3 (20,
30, 40 são os elementos nesses índices na tupla)

# Saída: (20, 30, 40)
```

```
# Saída: (20, 30, 40)
```

Exercício de Tupla

Crie uma tupla com 5 números e exiba o maior e o menor número usando funções nativas de Python.

```
# Criando uma tupla de números
tupla_numeros = (4, 10, -2, 5, 9)

# Encontrando o maior e o menor número
maior_numero = max(tupla_numeros)

menor_numero = min(tupla_numeros)

print(f"Maior número: {maior_numero}, Menor número: {menor_numero}")

# Saída: Maior número: 10, Menor número: -2
```

```
# Saída: Maior número: 10, Menor número: -2
```

```
print(f"Maior número: {maior_numero}, Menor número: {menor_numero}")
```

Exercício de Tupla

Crie uma tupla com 5 números e exiba o maior e o menor número usando funções nativas de Python.

- **Tupla:** A tupla `tupla_numeros` armazena cinco números diferentes. Uma tupla é uma estrutura de dados em Python que, ao contrário de listas, é imutável (não pode ser alterada após ser criada). As tuplas são definidas usando parênteses `( )` em vez de colchetes `[ ]` como nas listas.
- A função `max()` é uma função embutida no Python que retorna o maior valor dentro de uma sequência (nesse caso, a tupla).
- Da mesma forma, a função `min()` retorna o menor valor em uma sequência.

```
# Criando uma tupla de números
tupla_numeros = (4, 10, -2, 5, 9)

# Encontrando o maior e o menor número
maior_numero = max(tupla_numeros)

menor_numero = min(tupla_numeros)

print(f"Maior número: {maior_numero}, "
      f"Menor número: {menor_numero}")

# Saída: Maior número: 10, Menor número: -2
```

```
# 291q9: W9TOL uQW6LO: 10' W6UOL uQW6LO: -2
```

Exercícios



Exercício

Faça um programa que receba uma lista com 10 valores inteiros e guarde em uma lista. Após isso, armazene em outra lista apenas com números pares. Mostre na tela a nova lista com os valores recebidos.

Console

```
lista = []
lista_par = []

for num in range(10):
    numeros = int(input(f'Digite o {num + 1}º número: '))
    lista.append(numeros)

for item in lista:
    if item % 2 == 0:
        lista_par.append(item)

print(lista_par)
```

```
Digite o 1 numero: 1
Digite o 2 numero: 3
Digite o 3 numero: 8
Digite o 4 numero: 4
Digite o 5 numero: 6
Digite o 6 numero: 9
Digite o 7 numero: 10
Digite o 8 numero: 68
Digite o 9 numero: 22
Digite o 10 numero: 7
[8, 4, 6, 10, 68, 22]
```

Exercício

Dada uma lista com os seguintes valores:

```
lista = ['banana', 'laranja', 'maçã', 'morango', 'graviola']
```

Qual fruta está na posição 3?

Faça o comando que busque a posição 3 e mostre na tela o resultado.

Console

```
lista = ['banana', 'laranja', 'morango', 'abacaxi']  
print(lista[3]) # Acessa o quarto elemento da lista
```

morango

Exercício

Faça um programa que receba uma tupla e realize a adição de um elemento fazendo a conversão. Mostre na tela a lista e a tupla modificada.

1. Criar uma lista
2. Adicionar os números na lista
3. Depois converter a lista em tupla
4. Mostrar os dois na tela

```
lista = []
tupla = ()

for i in range(4):
    n = int(input('Informe o número: '))
    lista.append(n)

print(lista)

tupla = tuple(lista)

print(tupla)
```

Console

```
Informe o número: 8
Informe o número: 5
Informe o número: 5
Informe o número: 6
[8, 5, 5, 6]
(8, 5, 5, 6)
```